

S3データ不着検知時のクライアントへのメール通知 ーテスト項目

- **SPEC:** client-email-notification@0.4.0
- **TASK rev:** 3
- **TEST rev:** 3

方針

- テストケースは TASK と 1:1 対応させる (TC-00N ↔ TASK-00N)
- 検証の起点は SPEC の AC であり、実装コードの内部詳細ではない
- 外部依存 (SES API, Discord Webhook, ファイルシステム) は unit テストでは mock/stub する
- e2e・手動テストは実環境で一度だけ確認し、反復 CI に含めない

テスト実行戦略

フェーズ	実行内容	頻度
CI on PR	unit + static	PR 毎
CI on merge	unit + static + integration	merge 毎
手動	e2e (実環境)	デプロイ時

TC-001: Message dataclass と Channel 基底クラス

- **TASK:** TASK-001
- **REQ:** REQ-002
- **種別:** static
- **実行:** mypy / pytest

検証項目

AC	検証内容	手順
—	<code>Message</code> が frozen dataclass であり、全フィールドが型アノテーション付きで定義されている	<code>mypy src/channels/message.py</code> がエラーなしで通過する
—	<code>Channel</code> が ABC であり、 <code>id</code> (property) と <code>send(Message)</code> が抽象メソッドとして定義されている	<code>mypy src/channels/base.py</code> がエラーなしで通過する
—	<code>Message</code> インスタンスの属性を変更しようとする と <code>FrozenInstanceError</code> が発生する	<code>pytest: msg.title = "x"</code> が <code>FrozenInstanceError</code> を送出する

TC-002: DiscordChannel の embed 構築と送信

- **TASK:** TASK-002
- **REQ:** REQ-002
- **種別:** unit + static
- **実行:** pytest + webhook mock / grep

検証項目

AC	検証内容	手順
AC-002-1 (部分)	<code>DiscordChannel.send(message)</code> が Discord Webhook API を呼ぶ	<code>DiscordWebhook.execute</code> を mock し呼ばれたことを確認
—	<code>_to_embed()</code> が <code>message.severity</code> に対応する color を設定する	severity = HIGH / MEDIUM / LOW それぞれで embed の color 値を確認
—	embed の title に <code>message.title</code> が含まれる	embed.title が <code>message.title</code> と一致する
—	embed の footer に <code>message.alarm_name</code> が含まれる	embed.footer が <code>message.alarm_name</code> を含む
—	main.py から <code>DiscordWebhook</code> / <code>DiscordEmbed</code> の直接 import・インスタンス化がない	grep " <code>DiscordWebhook</code> <code>DiscordEmbed</code> " src/main.py が 0 件 (<code>_post_minimal_embed</code> / <code>_post_prompt_attachment</code> は除く)
—	<code>DISCORD_SEVERITY_COLOR</code> 定数が main.py に存在せず discord.py に存在する	grep " <code>DISCORD_SEVERITY_COLOR</code> " src/main.py が 0 件 / src/channels/discord.py に定義あり

TC-003: error-routing.yml の読み込みとフォールバック、error_id 判定

- **TASK:** TASK-003
- **REQ:** REQ-005, REQ-007
- **種別:** unit
- **実行:** pytest

検証項目

AC	検証内容	手順
AC-005-1	<code>_load_error_routing()</code> が <code>{error_id: entry}</code> 形式の辞書を返す	YAML ファイルをロードし <code>result["s3_data_missing"]</code> <code>["channels"]</code> にメールチャネルが含まれる

AC	検証内容	手順
AC-005-2	<code>_resolve_channel_ids("unknown_error", routing)</code> が <code>["discord"]</code> を返す	戻り値が <code>["discord"]</code> 、 <code>logger.warning</code> が呼ばれた
AC-005-1	<code>_resolve_channel_ids("s3_data_missing", routing)</code> がメールチャンネルを含むリストを返す	戻り値に <code>"email.dev"</code> が含まれる
AC-007-1	<code>_resolve_error_id("hdw-sakura", [])</code> が <code>"s3_data_missing"</code> を返す	戻り値が <code>"s3_data_missing"</code>
AC-007-1b	<code>_resolve_error_id("hdw-sakura", [log_row])</code> が <code>"lambda_failure"</code> を返す	戻り値が <code>"lambda_failure"</code>
AC-007-2	<code>_resolve_error_id("unknown-alarm", [])</code> が <code>"unknown"</code> を返し WARNING ログが出る	戻り値が <code>"unknown"</code> 、 <code>logger.warning</code> が呼ばれた

TC-004: resolve_addresses の境界条件

- **TASK:** TASK-004
- **REQ:** REQ-006
- **種別:** unit
- **実行:** pytest

検証項目

AC	検証内容	手順
AC-006-1	group_id に対応する add リストが返る	<code>entries = [{"id": "dev", "add": ["a@x.com"]}]</code> 、 <code>resolve_addresses("dev", entries) → ["a@x.com"]</code>
AC-006-2	<code>email.dev → group_id dev → id: dev</code> エントリの add が使われる	上記 entries で結果に <code>"a@x.com"</code> が含まれる
AC-006-3	group_id が存在しない場合、空リストを返し WARNING ログが出る	<code>resolve_addresses("missing", entries) → []</code> 、 <code>logger.warning</code> が呼ばれた
—	email.yaml がリスト of dict 形式で正しくロードされる	<code>yaml.safe_load</code> の結果が <code>list[dict]</code> 型である

TC-005: SESEmailChannel の SES API 呼び出し

- **TASK:** TASK-005

- **REQ:** REQ-001, REQ-002
- **種別:** unit
- **実行:** pytest + boto3 mock (moto または unittest.mock)

検証項目

AC	検証内容	手順
—	<code>SESEmailChannel("dev").id</code> が <code>"email.dev"</code> を返す	プロパティの戻り値を確認
AC-001-1 (間接)	<code>SESEmailChannel.send()</code> が <code>boto3.client("ses").send_email</code> を呼ぶ	boto3 を mock し <code>send_email</code> が呼ばれたことを確認
AC-002-1	<code>send_email</code> の <code>Destination.ToAddresses</code> が <code>resolve_addresses</code> の戻り値と一致する	mock の呼び出し引数を確認
AC-006-3 (間接)	<code>group_id</code> が <code>email.yaml</code> に存在しない場合、 <code>send_email</code> は呼ばれない	groups に <code>group_id</code> なし → <code>send_email</code> 未呼び出し、WARNING ログあり
AC-001-3	<code>Source</code> が <code>SES_FROM_ADDRESS</code> 環境変数の値と一致する	env var を設定して <code>send_email.Source</code> を確認

TC-006: HTML・プレーンテキストテンプレート

- **TASK:** TASK-006
- **REQ:** REQ-003, REQ-004
- **種別:** unit
- **実行:** pytest

検証項目

AC	検証内容	手順
AC-004-1	<code>send_email</code> リクエストに <code>Body.Html</code> と <code>Body.Text</code> の両方が含まれる	mock の呼び出し引数に <code>Body.Html.Data</code> と <code>Body.Text.Data</code> が存在する
AC-003-1 / AC-004-2	<code>_to_html()</code> の戻り値に <code>message.ship_name</code> が含まれる	文字列に <code>ship_name</code> 値が含まれる
AC-003-2 / AC-004-2	<code>_to_html()</code> の戻り値に JST 形式のタイムスタンプが含まれる	文字列に <code>"JST"</code> と年月日時分が含まれる
AC-004-2	<code>_to_html()</code> の戻り値に <code>message.severity</code> が含まれる	文字列に <code>"HIGH"</code> / <code>"MEDIUM"</code> / <code>"LOW"</code> が含まれる

AC	検証内容	手順
AC-004-2	<code>_to_html()</code> の返り値に <code>message.actions</code> の全アクションが含まれる	各 action 文字列が返り値に含まれる
AC-003-1 / AC-003-2	<code>_to_plain()</code> にも同内容 (ship_name・時刻・severity・actions) が含まれる	同様のアサーションをプレーンテキストに適用

TC-007: `_dispatch` ルーティングアルゴリズム

- **TASK:** TASK-007
- **REQ:** REQ-002, REQ-005, REQ-007
- **種別:** integration + static
- **実行:** pytest (DiscordChannel / SESEmailChannel を mock) / grep

検証項目

main.py の静的確認

AC	検証内容	手順
—	main.py に <code>Message</code> 構築コードが存在する	<code>grep "Message(" src/main.py</code> がヒットする
—	main.py に <code>_dispatch(alarm_name, message, error_id)</code> 呼び出しが存在する	<code>grep "_dispatch" src/main.py</code> がヒットする
—	main.py に <code>_resolve_error_id(alarm_name, log_rows)</code> 呼び出しが存在する	<code>grep "_resolve_error_id" src/main.py</code> がヒットする
—	「ログ 0 件の早期 return」ブロック (旧 lines 649–662) が削除されている	<code>grep "no_logs" src/main.py</code> が 0 件
—	Bedrock 呼び出し後パスに <code>DiscordWebhook</code> (/ <code>DiscordEmbed</code> の直接インスタンス化がない	対象ブロックに該当コードがない (<code>_post_minimal_embed</code> ・ <code>_post_prompt_attachment</code> は除く)

report → Message フィールドマッピングの確認

フィールド	期待値	手順
<code>title</code>	<code>report["summary"]</code> と一致	<code>_normalize_report()</code> の出力で Message 構築し <code>title</code> を確認
<code>actions</code>	<code>report["suggested_actions"]</code> と一致	list の内容・順序を確認
<code>timestamp</code>	Alarm 発火時刻の datetime (UTC aware)	<code>center</code> と同値、 <code>tzinfo</code> が UTC
<code>ship_name</code>	<code>_extract_ship_name_from_alarm_name()</code> の結果	"hdw-sakura" → "sakura"

email.* channel_id の解析とレジストリ構築

AC	検証内容	手順
AC-006-2	<code>_build_channel_registry(["discord", "email.dev"])</code> が <code>"email.dev"</code> キーに <code>SESEmailChannel</code> を生成する	<code>registry["email.dev"]</code> が <code>SESEmailChannel</code> のインスタンスであることを確認
—	<code>SESEmailChannel</code> の <code>group_id</code> が <code>"dev"</code> である	<code>channel._group_id == "dev"</code> を確認
AC-007-3	<code>"email."</code> で始まらない未知の <code>channel_id</code> はレジストリに登録されない	<code>registry.get("unknown_channel")</code> が <code>None</code>

ルーティング・フォールバック動作

AC	検証内容	手順
AC-005-1	<code>_dispatch</code> が routing の <code>channels</code> に従って全チャンネルに送信する	<code>error_id="s3_data_missing", channels: ["discord", "email.dev"]</code> → 両方の <code>send</code> が呼ばれた
AC-005-1	<code>_dispatch</code> が <code>error_id="lambda_failure"</code> でも全チャンネルに送信する	<code>channels: ["discord", "email.dev"]</code> → 両方の <code>send</code> が呼ばれた
AC-007-2	<code>error_id</code> が routing に存在しない場合、 <code>discord</code> のみに送信し WARNING が出る	<code>error_id = "unknown"</code> → <code>DiscordChannel.send</code> のみ呼ばれた、WARNING あり
AC-007-3	未知の <code>channel_id</code> はスキップし WARNING が出る	routing に <code>"other_channel"</code> を追加 → WARNING あり、例外なし
AC-002-2 / AC-007-4	<code>SESEmailChannel.send()</code> が例外を発生させても <code>DiscordChannel.send()</code> が呼ばれる	<code>SESEmailChannel</code> を例外送出 mock に設定 → <code>DiscordChannel.send</code> が呼ばれた
AC-007-4	<code>DiscordChannel.send()</code> が例外を発生させても <code>SESEmailChannel.send()</code> が呼ばれる	<code>DiscordChannel</code> を例外送出 mock に設定 → <code>SESEmailChannel.send</code> が呼ばれた
AC-007-5	いかなるフォールバック・例外発生時も <code>_dispatch</code> が例外を送出しない	全 channel を例外送出 mock に設定して <code>_dispatch</code> が正常終了（例外なし）

TC-008: IAM ポリシーと SES セットアップ（手動確認）

- **TASK:** TASK-008
- **REQ:** REQ-001
- **種別:** e2e（手動）

- **実行:** AWS コンソール / CLI で確認

確認手順

AC	確認内容	確認方法
AC-001-1	Lambda 実行ロールに <code>ses:SendEmail</code> / <code>ses:SendRawEmail</code> が付与されている	IAM コンソールでロールのポリシーを確認、または <code>aws iam get-role-policy</code>
AC-001-2	SES サンドボックスが解除されており、外部アドレスへ送信できる	SES コンソールの「Account dashboard」でサンドボックス解除状態を確認
AC-001-3	送信元アドレスが <code>@scrumsign.com</code> ドメインである	<code>aws ses get-identity-verification-attributes --identities scrumsign.com</code> で検証済みを確認
—	実際にテストメールが届く	<code>aws ses send-email --from alerts@scrumsign.com --to <自分のアドレス> ...</code> でメール受信を確認

TC-009: Lambda コンテナイメージへの config ファイル包含

- **TASK:** TASK-009
- **REQ:** REQ-005, REQ-006
- **種別:** static (Docker ビルド後確認)
- **実行:** docker build + docker run

確認手順

AC	確認内容	手順
AC-005-1 (間接)	<code>config/error-routing.yml</code> が Lambda イメージに含まれる	<code>docker run --rm <image> ls /var/task/config/</code> に <code>error-routing.yml</code> が含まれる
AC-006-1 (間接)	<code>config/email.yaml</code> が Lambda イメージに含まれる	<code>docker run --rm <image> ls /var/task/config/</code> に <code>email.yaml</code> が含まれる
—	イメージ内の両ファイルが読み込み可能な YAML である	<code>docker run --rm <image> python -c "import yaml; yaml.safe_load(open('/var/task/config/error-routing.yml'))"</code> が正常終了